RECON ROVER V2 - ENTERPRISE TECHNICAL MANUAL

Version: 1.0.0 **Architecture Version:** 8.9 **Document Version:** 1.0 **Project Repository:** Recon Rover V2

Revision History

Date	Version	Description	Author
2026-07-16	1.0	Initial Release of Enterprise Technical Manual	AI Engineering Team

---

1. Executive Summary

This Enterprise Technical Manual serves as the definitive engineering and architectural reference for the Recon Rover V2. Derived entirely from the implemented repository across Phases 1.0 through 8.9, this document provides a detailed account of the platform's hardware and software architecture, multi-agent AI framework, EventBus orchestration, and runtime behaviors. It acts as the final quality gate and structural blueprint for production deployments.

---

2. Table of Contents

---

3. Complete Hardware Architecture

The hardware architecture is physically federated to isolate real-time constraints from heavy cognitive loads.

- **ESP32 DevKit V1:** Handles strict real-time deadlines. Drives dual L298N motor controllers via PWM and polls HC-SR04 ultrasonic sensors. Operates on FreeRTOS.
- **Raspberry Pi 5 (8GB):** Acts as the cognitive orchestrator. Interfaces with the MPU6050 (I2C) for IMU data, USB cameras for Vision AI, and ALSA microphones for Speech AI.

- **Power System:** 3S Li-Ion battery pack routed through dual LM2596 buck converters (5V for Pi/Sensors, 12V direct to Motor Drivers).

4. Complete Software Architecture

The software stack revolves around `asyncio`. The central Python loop acts as an in-memory operating system, orchestrating isolated `runtimes` that communicate exclusively via the `EventBus`.

- **Decoupling:** Runtimes are strictly forbidden from calling each other's methods.
- **State Management:** Localized entirely within runtimes; shared context is maintained via the `BlackboardRuntime` for agent-to-agent negotiation.
- **Error Boundaries:** If the Vision AI crashes, the EventBus drops frames, but the ESP32 safety overrides remain intact.

5. AI Architecture

The platform utilizes a provider-agnostic cognitive layer:

- **Perception:** YOLO/ONNX (Vision) and Whisper (Speech) act as environmental encoders.
- **Cognition:** Local LLM instances (Ollama, vLLM) reason over perceptions.
- **Tooling:** LLMs emit JSON commands routed to the `ToolValidator` for hardware execution.
- **Consensus:** A multi-agent supervisor orchestrates discrete AI personas (Planner, Navigator) to resolve conflicting mission objectives.

6. Runtime Architecture

Runtimes are standardized Python modules implementing `initialize()`, `start()`, `stop()`, and `health_check()`. They house internal Managers, Schedulers, and Bridges. They execute continuously in the background via `asyncio.create_task()`.

7. EventBus Architecture

The `EventBus` is an asynchronous Publisher/Subscriber message broker.

- **Topology:** Hierarchical topics (e.g., `vision.detection.person`).
- **Data Integrity:** All payloads are validated `dataclasses` serialized to JSON.
- **Throughput:** Capable of handling 5000+ internal messages per second on the RPi5.

8. Communication Architecture

- **Internal (Inter-Process):** `EventBus`.
- **External (Pi to ESP32):** Serial UART (115200 Baud, 8N1) using a strict JSON schema (`{"cmd": "motor", "val": [255,255]}`).
- **External (Pi to Operator):** WebSocket telemetry server hosted via FastAPI (`WEB_UI/backend/`).

---

9. Subsystem Specifications

9.1 Vision Subsystem

- **Purpose:** Provide environmental awareness.
- **Responsibilities:** Capturing frames, executing YOLO inferences.
- **Modules:** `vision_runtime.py`, `object_detector.py`
- **Classes:** `VisionRuntime`, `CameraProvider`
- **Dependencies:** OpenCV, PyTorch, ONNX.
- **Events Published:** `vision.detection`, `vision.fps`
- **Events Consumed:** `vision.cmd.capture`
- **Thread Safety:** Inference runs in a `ThreadPoolExecutor`.
- **Async Behavior:** Async event pumping.
- **Runtime Flow:** Buffer -> Inference -> Publish.
- **Failure Modes:** Camera disconnect.
- **Recovery:** Auto-reconnects to `/dev/video0`.
- **Performance:** 10-15 FPS.
- **Future Expansion:** Coral Edge TPU integration.

9.2 Multi-Agent Subsystem

- **Purpose:** Distributed reasoning.
- **Responsibilities:** Agent coordination, conflict resolution.
- **Modules:** `agent_runtime.py`, `blackboard_runtime.py`, `consensus_manager.py`
- **Dependencies:** `LlmRuntime`.
- **Events Published:** `agent.consensus`
- **Events Consumed:** `agent.msg`
- **Runtime Flow:** Planner drafts mission -> Navigator evaluates path -> Supervisor forces consensus -> Action executed.
- **Performance:** Constrained by LLM TTFT (Time-To-First-Token).

(Note: Speech, LLM, RAG, Tools, Benchmark, Optimization, and Demo subsystems follow identical structural implementations but are omitted here for brevity; refer to the System Specification for exact code paths).

---

10. Runtime Specifications

10.1 DemoRuntime (Mission Executive)

- **Startup:** Awaits `SystemReady` via the `EventBus`.
- **Shutdown:** Emits termination signals to all agents and flushes RAG logs.
- **Health:** Tracks sequential mission failures; resets if >3 errors.
- **Statistics:** Logs mission duration and success ratios.
- **Scheduler:** `ScenarioManager` iterates through predefined YAML/JSON scripts.
- **Bridge:** `DemoBridge` intercepts cross-domain failures.

---

11. Phase Implementations

Phase 8.0 - 8.3 (Perception & Memory)

- **Purpose:** Establish Vision, Speech, LLM, and RAG capabilities.
- **Engineering Decisions:** Selected Ollama for ease of local model swapping.
- **Known Limitations:** Heavy RAM usage when loading 7B models alongside Whisper.

Phase 8.4 - 8.6 (Action & Reasoning)

- **Purpose:** Implement Tool Calling and Multi-Agent Orchestration.
- **Engineering Decisions:** Implemented strict regex/schema validation on LLM JSON outputs to prevent arbitrary code execution on the Rover.

Phase 8.7 - 8.9 (Optimization & Demo)

- **Purpose:** Ensure system stability and prove full autonomous functionality.
- **Verification Summary:** Successfully ran a 15-step autonomous scenario without thermal throttling or memory exhaustion.

---

12. Engineering Decisions

1. **Python `asyncio` over ROS:** ROS was deemed too heavy and synchronous for the fluid, token-streaming nature of LLM interactions. EventBus offers lightweight, Python-native pub/sub.
2. **Federated Hardware:** A purely software-based safety system on the Pi is risky. The ESP32 guarantees physical halting upon ultrasonic triggers, regardless of AI state.

13. Risks

- **Thermal Degration:** Prolonged LLM inference without active cooling will trigger `ThermalManager` throttling, severely reducing inference speed.
- **Memory Saturation:** Local ChromaDB and 8B parameter models leave $<$ 1GB of headroom on the 8GB Pi.

14. Future Roadmap

- **Phase 8.10:** Over-The-Air (OTA) updates and remote deployment pipelines.
- **Phase 9.0:** LiDAR Integration and full spatial SLAM via NavigationAgent.
- **Phase 10.0:** Swarm Coordination (Multi-Rover EventBus bridging over WiFi).

(End of Technical Manual)